

## 1. What is File Handling?

**File handling** means reading from and writing to files stored on your computer using Python programs.

It allows you to:

- **Store data permanently** (unlike variables, which are temporary)
- **Access** and **modify** text or binary files
- **Automate tasks** such as log generation, data saving, or report creation

---

## 2. Types of Files

There are mainly two types of files:

| Type                | Description                                     | Example          |
|---------------------|-------------------------------------------------|------------------|
| <b>Text files</b>   | Store data in human-readable form               | .txt, .csv, .py  |
| <b>Binary files</b> | Store data in encoded form (not human-readable) | .jpg, .mp3, .dat |

We'll focus on **text files** here.

---

## 3. Opening a File in Python

To work with a file, you must first **open** it using the built-in `open()` function.

**Syntax:**

```
file = open(filename, mode)
```

**Common Modes:**

| Mode | Description                                                               |
|------|---------------------------------------------------------------------------|
| 'r'  | Read (default). Opens the file for reading only. Error if file not found. |
| 'w'  | Write. Creates a new file or overwrites existing file.                    |
| 'a'  | Append. Adds new data at the end of file.                                 |
| 'r+' | Read and write (must exist).                                              |
| 'w+' | Write and read (creates new or overwrites).                               |
| 'a+' | Append and read.                                                          |

---

## 4. Closing a File

After working with a file, you should **close** it to free system resources.

```
file.close()
```

But instead of doing this manually every time, Python offers a better way using the with statement.

---

## 5. Using with Statement (Best Practice)

Using with automatically opens and closes the file.

```
with open("data.txt", "r") as file:
```

```
    content = file.read()
```

```
    print(content)
```

```
# File automatically closed here
```

---

## 6. Reading from a File

There are three main ways to read file content.

### a) read() — Reads the entire file

```
with open("sample.txt", "r") as f:
```

```
    data = f.read()
```

```
    print(data)
```

#### Output (if file contains):

Hello Aditya

Welcome to Python File Handling

---

### b) readline() — Reads one line at a time

```
with open("sample.txt", "r") as f:
```

```
    line1 = f.readline()
```

```
    line2 = f.readline()
```

```
    print(line1)
```

```
    print(line2)
```

---

### c) readlines() — Reads all lines into a list

```
with open("sample.txt", "r") as f:
```

```
    lines = f.readlines()
```

```
print(lines)
```

**Output:**

```
['Hello Aditya\n', 'Welcome to Python File Handling\n']
```

---

## 7. Writing to a File

When writing, use mode 'w' or 'a'.

### a) Using 'w' (write mode)

It creates a new file or overwrites existing content.

with open("sample.txt", "w") as f:

```
f.write("This is the first line.\n")
```

```
f.write("This will overwrite old content.\n")
```

---

### b) Using 'a' (append mode)

It adds new data at the end of the file.

with open("sample.txt", "a") as f:

```
f.write("This is an appended line.\n")
```

---

## 8. Reading and Writing Together

Use 'r+', 'w+', or 'a+' modes.

Example:

with open("data.txt", "w+") as f:

```
f.write("Learning File Handling\n")
```

```
f.seek(0) # Move cursor to the beginning
```

```
print(f.read()) # Read content
```

---

## 9. Working with File Paths

You can use **absolute paths** or **relative paths**.

# Absolute path example

```
f = open("C:/Users/Aditya/Documents/file.txt", "r")
```

# Relative path example (same folder as your program)

```
f = open("file.txt", "r")
```

---

## 10. Checking if File Exists

You can use the os module to check file existence.

```
import os
```

```
if os.path.exists("sample.txt"):
```

```
    print("File exists")
```

```
else:
```

```
    print("File not found")
```

---

## 11. Example: Complete Program

# Writing data to a file

```
with open("notes.txt", "w") as f:
```

```
    f.write("Python File Handling Example\n")
```

```
    f.write("We are writing data into a file.\n")
```

# Reading data back

```
with open("notes.txt", "r") as f:
```

```
    content = f.read()
```

```
    print("File Content:\n", content)
```

# Appending new data

```
with open("notes.txt", "a") as f:
```

```
    f.write("Now we are appending new text.\n")
```

# Reading line by line

```
with open("notes.txt", "r") as f:
```

```
    for line in f:
```

```
print("Line:", line.strip())
```

---

## 12. Common File Handling Errors

| Error             | Cause                                                      |
|-------------------|------------------------------------------------------------|
| FileNotFoundError | File does not exist while opening in read mode             |
| PermissionError   | You don't have permission to open or modify the file       |
| IOError           | Input/output operation failed (like reading a closed file) |

---

## 13. Key Functions Summary

| Function     | Purpose                         |
|--------------|---------------------------------|
| open()       | Opens a file                    |
| read()       | Reads entire file               |
| readline()   | Reads one line                  |
| readlines()  | Reads all lines as list         |
| write()      | Writes a string to file         |
| writelines() | Writes list of strings          |
| seek()       | Moves the file cursor           |
| tell()       | Returns current cursor position |
| close()      | Closes the file                 |

---

## 14. Example: tell() and seek()

with open("demo.txt", "w+") as f:

```
f.write("ABCDEFGF")
```

```
print("Current position:", f.tell()) # Cursor at end
```

```
f.seek(2) # Move cursor to 3rd character
```

```
print("After seek, position:", f.tell())
```

```
f.write("XYZ")
```

Resulting file content:

ABXYZFG

---

## 15. Summary

| Concept        | Description                                       |
|----------------|---------------------------------------------------|
| File Handling  | Reading, writing, or modifying files using Python |
| Main Functions | open(), read(), write(), close()                  |
| Best Practice  | Always use with open() for automatic closing      |
| Modes          | 'r', 'w', 'a', 'r+', 'w+', 'a+'                   |
| Modules        | os for file management                            |

---